

Immutable datatype

Here we can't change original data object but can do modification.

Eg: String, tuple and single value datatype.

Tuple Data Type ^{Another name of Tuple is Composite key}

- It is a collection of homogeneous and heterogeneous type of data, separated by comma.

Note: In tuple boundary condition is not mandatory but

- The boundary condition comma is mandatory ('(el1, el2, el3)') (E.g. 1)

- It can have duplicate values.

- It is ordered (it will return o/p in same order)

- length `len(variable name)` return number of elements present inside a tuple.

- It is also immutable data type (original value will not change)

- Indexing & slicing can be done on tuple

Homogeneous :- Same data type elements.

a = ("hello", "apple", "boy", "good")

Heterogeneous :- (Different data type elements)
 $t = (12, 42.6, \text{True}, "hello", [1, 2, 3])$

How to create tuple data type :-

1) Direct Assignment :-

$a = (12, 42.6, "hello", 75.23)$

2) By using class object :-

$a = \text{tuple}((1, 2, 3))$
 $a(1, 2, 3)$

How to create empty tuple ?

$a = ()$

o/p :- $()$

How to create empty tuple using Constructor / Object ?

$b = \text{tuple}()$

o/p :- $()$

How to create a single value tuple ?

$c = (1,)$

o/p :- $(1,)$

$\text{type}(c)$

o/p :- $\langle \text{class 'tuple'} \rangle$

Object

Tuple Methods :

Finding attributes in tuple data type.

(dir(tuple))

1. Count()

2. Index()

1. Count()

- It will return number of occurrence of the element.
- If specified value is not there it will return 0.

Syntax :-

Vn.count (element)

t = (45, 42, 78, 42)

t.count(42)

2

t.count(45)

1

2. Index()

- Index will return index value of specified element. $\textcircled{C} \rightarrow \textcircled{P}$
- It will always return index number of 1st occurrence of specified value.

If the value is not present it will return value error.

Syntax :- `vn.index (element)`

`t = (45, 42, 78, 12, 42, 45, 12, 45)`
`t.index (45)`
 0

`t.index (45, 7, 9)`

Indexing of Tuple.

Syntax :- `vn [index number]`

`t = ("hello", "hey", 12.2, 42, "good", 45, [12, 45, 75])`

`t[4]`

'good'

`t[6]`

`[12, 45, 75]`

`t[6][1]`

45

`t[4][1]`

'o'

Question

1) What is difference between find and index method.

2) What is difference between indexing & Slicing.